

# 一個啟發式演算法在瓶頸選擇內部節點最小生成樹問題上

## A heuristic algorithm for the bottleneck selected-internal minimum spanning tree problem

Chien Yuan Ho<sup>1</sup>, Yu Ting Kuo<sup>2</sup>, Yen Hung Chen<sup>3</sup>

Department of Computer Science,  
Taipei Municipal University of Education, Taiwan  
[h10292@hotmail.com](mailto:h10292@hotmail.com)<sup>1</sup>, [kuo5400@hotmail.com](mailto:kuo5400@hotmail.com)<sup>2</sup>  
[yhchen@tmue.edu.tw](mailto:yhchen@tmue.edu.tw)<sup>3</sup>

### 摘要

本論文主要是研究一個組合最佳化的問題：瓶頸選擇內部節點最小生成樹問題 (bottleneck selected-internal minimum spanning tree problem)。給定一個無向完全圖  $G=(V,E)$ ，一個非負數的權重函數在邊上及一個節點的集合  $R \subseteq V$ ，選擇內部節點生成樹為一棵生成樹連通  $V$  內的所有節點，但是在  $R$  內的節點不能是樹葉 ( $|V \setminus R| \geq 2$ )。瓶頸選擇內部節點最小生成樹問題定義為找出一棵選擇內部節點最小生成樹使得樹上最大邊的權重要最小。本研究首先證明瓶頸選擇內部節點最小生成樹問題為 NP-Complete 接著設計了一個啟發式演算法來解決此問題，並模擬在一些亂數產生的資料上。實驗結果顯示我們的啟發式演算法所得的解與最佳解間的倍率大約為 1.3 至 1.4 之間，最高到 1.48 倍。在執行效率上，我們的啟發式演算法所用之時間平均為 1 至 5 毫秒 (ms)。

### 1 緒論

最小生成樹 (minimum spanning tree, MST) 問題是圖論中相當重要且基礎的研究。一棵最小生成樹可應用在網路設計 (Networks Design) [8] 及當做許多組合問題的近似解 [8,25]。兩個相當有名的多項式時間的正確演算法 (exact algorithm) 來解決 MST 問題分別是 Prim 及 Kruskal [8]。

一個 MST 問題的變形問題稱為斯坦納樹問題 (Steiner tree problem)。給定一個無向圖  $G=(V,E)$ ，一個非負數的權重函數在邊上及一個節點集合  $R \subseteq V$ ，一棵斯坦納樹 (Steiner Tree) 定義為圖上的一棵樹連通  $R$  內的所有節點。斯坦納樹問題 (STP) 為在圖上找一棵權重加總最小的斯坦納樹 [12,18]。STP 有許多方面的應用，例如群播繞境 (Multicast routing) [5,6,9,10,15]、生物上的演化樹之重建 (Evolutionary tree reconstruction in biology) [6,15,21] 與大型積體電路繞線 (VLSI routing) [4,6,9,15,16]。然而 STP 已被證明為 NP-Complete [11] 並有相當多的近似演算法被提出

[1,3,10,13,15,23,24,26,27,28]。目前最好的近似率為  $1 + (\ln 3/2) \approx 1.39$  [3]。

基於 Sensor network allocation 應用，謝孫源教授與楊仕丞定義了選擇內部節點斯坦納樹問題 (selected-internal Steiner tree problem) [14]。給定無向完全圖  $G=(V,E)$ ，一個非負數的權重函數在邊上及有兩個節點集合  $R$  與  $R'$ ， $R \subseteq V$  且  $R' \subseteq R$ 。選擇內部節點斯坦納樹問題 (SISTP) 為在圖上找一棵權重加總最小的斯坦納樹連通  $R$  內所有節點但要求  $R'$  內的節點不能是樹葉。謝孫源教授與楊仕丞證明 SISTP 為 NP-Complete 並提出  $2\rho$  倍的近似演算法 [14]，其中  $\rho$  是 STP 中最好的近似率 (目前約 1.39) [3]。Li 等人設計一個近似演算法改善此問題的近似率達到  $1+\rho$  [19]。Li 等人同時提出了一個生成樹的版本，命名為選擇內部節點最小生成樹問題 (selected-internal minimum spanning tree problem) [19]。給定一個無向完全圖  $G=(V,E)$ ，一個非負數的權重函數 (cost function) 在邊上及一個節點的集合  $R \subseteq V$ ，選擇內部節點生成樹定義為一棵生成樹連通  $V$  內的所有節點，但是在  $R$  內的節點不能是樹葉 ( $|V \setminus R| \geq 2$ )。選擇內部節點最小生成樹問題 (SIMSTP) 定義為找出一棵選擇內部節點最小生成樹其權重加總要最小 [19]。Li 等人 [19] 證明此問題為 NP-Complete 並提出一個兩倍的近似演算法解決 SIMSTP，時間複雜度為  $O(|V|^4)$ 。Liu 等人 [20] 提出了一個啟發式演算法解 SIMSTP，實驗結果顯示此啟發式演算法所得的解與最佳解間的倍率大約為 1.5 至 1.6 之間，最高到 1.8 倍。

雖然 SIMSTP 是定義為 min-sum 目的函數，一些網路及大型積體電路繞線 (routing) 則會考慮另外一種目的函數：min-max (bottleneck) [6, 9, 10, 15]。因此在計畫我們定義了一個 SIMSTP 的 min-max 版本稱為瓶頸選擇內部節點最小生成樹問題 (bottleneck selected-internal minimum spanning tree problem)。瓶頸選擇內部節點最小生成樹問題 (BSIMSTP) 定義為找出一棵選擇內部節點生成樹使得樹上最大邊的權重要最小。BSIMSTP 截至目前為止尚無相關文獻在探討此問題。因此，BSIMSTP 的問題難度及相關的演算法是沒有被分析及設計過的。基於 SISTP

可應用在 multicast resource allocation [16]，因為 SIMSTP 為 SISTP 的生成樹版本，所以 SIMSTP 即 BSIMSTP 可應用在廣播資源配置(broadcast resource allocation)。底下我們簡略的說明 SIMSTP 及 BSIMSTP 的應用。

廣播資源配置 (Broadcast Resource Allocation): 在一個 network routing 中，傳送者需要把訊息傳遞給網路的所有接收者，此廣播方式有時會以造出一棵最小生成樹方式來將傳送者的訊息傳遞給所有的接收者。然而某些在網路中特殊的裝置(伺服器)必須被當作轉送塔(transmitter)，傳送塔為轉送站，因此必須是樹內的內部節點 [16]。網路內的其他的裝置則無此限制。我們的目的是要找到一棵權重最小的生成樹，因此在網路的廣播資源配置問題即可轉換到 SIMSTP 上。當我們要求的是一棵最大權重要最小的生成樹時，網路的廣播資源配置問題立即可轉換到 BSIMSTP 上。

在此論文中，我們首先證明 BSIMSTP 為 NP-complete，接著我們設計了一個啟發式演算法(heuristic algorithm)解決 BSIMSTP。此方法是利用在 metric graph(完全圖邊常滿足三角不等式)上的瓶頸旅行業務員問題 (Bottleneck Traveling Salesman Problem, BTSP) 之 2 倍的近似演算法 [22]改良後得到。其時間複雜度為  $O(|V|^4)$ 。接著我們實作此一演算法並對一些亂數產生的資料進行模擬。實驗結果顯示我們的演算法所得到的解的權重與最佳解的權重的倍率將不會超過 1.5 倍。在執行效率上，我們的演算法所需之時間平均約為 1 至 5 毫秒(ms)。

本論文第二章我們將證明 BSIMSTP 為 NP-Complete。第三章我們提出一個啟發式演算法來解決 BSIMSTP 並分析其時間複雜度。第四章利用我們的啟發式演算法對一些亂數產生的資料進行模擬(simulation)測試，藉此結果說明我們的方法之優劣。最後在第五章說明 BSIMSTP 的未來研究方向。

## 2 BSIMSTP 為 NP-Complete

本章中，我們首先證明 BSIMSTP 為 NP-Complete。轉換(reduction)方法是透過漢米爾頓路徑問題(Hamiltonian path problem)。米爾頓路徑問題是一個非常有名的 NP-Complete problem [8]描述如下。

### Hamiltonian Path Problem

Instance: 一個圖  $G(V,E)$  及兩個在  $V$  內 節點 (vertices)  $x$  及  $y$

Problem: 是否存在一條 path 從  $x$  到  $y$  經過圖  $G$  的每一點剛好一次?

為了證明 BSIMSTP 為 NP-Complete，我們定義 BSIMSTP 的決定版(decision version)。

### Bottleneck Selected-Internal Minimum Spanning tree Decision Problem (D-BSIMSTP)

Instance: 一個正整數  $m$ ，一個無向完全圖  $G=(V,E)$ ，一個非負數的權重函數  $c$  (cost function) 在邊上及一個節點的集合  $R \subset V$

Problem: 是否存在一個選擇內部節點生成樹對於  $R$  在圖  $G$  使得樹上最大邊的權重小於或等於  $m$ ?

Theorem 1. D-BSIMSTP 一個 NP-complete 問題，當 instance 是一個 metric graph

Proof: 首先，這是 D-BSIMSTP 是在 NP 內這是容易可得知的。接下來我們呈現這個轉換(reduction): 將米爾頓路徑問題轉換到 BSIMSTP 的決定版。令一個圖  $G=(V,E)$  和圖上的兩節點  $v_x, v_y$  是米爾頓路徑問題的 instance。我們建構一個 D-BSIMSTP 的 instance: 一個完全圖  $G'=(V,E')$  和一個非負數的權重函數  $c$  (cost function) 在邊上，一個節點的集合  $R \subset V$  及  $m$  定義如下：

$$m=1$$

$$R=V \setminus \{v_x, v_y\}$$

For each edge  $(u,v) \in E'$ ,

$$\begin{cases} c(u,v)=1, & \text{if } (u,v) \in E \\ c(u,v)=2, & \text{otherwise} \end{cases}$$

很明顯的圖  $G'$  滿足三角不等式。底下我們證明圖  $G$  有一個漢米爾頓路徑從  $v_x$  到  $v_y$  若且唯若(if and only if) 圖  $G$  有一個選擇內部節點生成樹其最大的邊為  $m$ 。

(Only if) 假設在圖  $G$  中有一個 Hamiltonian path 從  $v_x$  到  $v_y$ ，很清楚的這個 Hamiltonian path 是在  $G'$  中的一個選擇內部節點生成樹  $T$  對於  $R$ ，而且  $T$  的每個邊的權重為 1。

(If) 假設圖  $G'$  中的一個選擇內部節點生成樹  $T$  對於  $R$ ，而且  $T$  的每個邊的權重為 1，很清楚的因為  $R=V \setminus \{v_x, v_y\}$ ，因此生成樹  $T$  必需為一個 path 連通所有的  $R \setminus \{v_x, v_y\}$  並且  $v_x$  及  $v_y$  需為樹的樹葉。因此生成樹  $T$  為  $G$  中有一個 Hamiltonian path 從  $v_x$  到  $v_y$ 。□

## 3 啟發式演算法在 BSIMSTP 上

本章中，我們提出一個啟發式演算法來解 SIMSTP。一個簡單的觀察是如果我們在  $V \setminus R$  內挑出任兩節點分別為起點  $v_s$  及終點  $v_d$ ，接下來我們找出一個 path 從起點  $v_s$  到終點  $v_d$  並經過  $V$  內所有的節點一次，則此一 path 會是 BSIMSTP 的一個合理解(feasible solution)，因為  $R$  內的節點在此 path 中都不會是樹葉。而且此 path 經過所有的節點，即是一個漢米爾頓路徑(Hamiltonian path) [8]。因此我們的啟發式演算法是透過修改 2 倍 BTSP 的近似演算法 [22] 找出 BSIMSTP 的一個合理解。這樣的方法可以在多項式時間內完成，時間為  $O(|V|^2)$ ，且我們的演算法執行模擬後的近似率會低於 1.5 倍。

我們描述此演算法如下：

輸入：一個無向完全圖  $G=(V,E)$ ，每個邊上有非負數之權重函數(需滿足三角不等式)，以及一內部節點集合  $R \subset V$  ( $|V \setminus R| \geq 2$ )

輸出：建構一個在  $G$  內生成樹  $T$  使得每個  $R$  內的節點必須為此生成樹的內部節點

1. For  $V \setminus R$  內的所有節點對(任兩節點  $v_s$  及  $v_d$ ) do
  - 1.1 令一個圖  $G_i$  是  $G$  的子圖 induced 藉由  $V \setminus \{v_s, v_d\}$ ，利用 BTSP 的 2 倍近似演算法 [22] 找出圖  $G_i$  中的 TSP 解。
  - 1.2 移除 TSP 中最短的一個邊(假設此邊為  $(v_a, v_b)$ )後，加入邊  $(v_s, v_a)$  及  $(v_b, v_d)$  形成一棵樹  $T'$ ，計算此生成樹  $T'$  的權重。
2. 找出依步驟 1 產生的所有生成樹中權重最小的一棵生成樹  $T$ 。

步驟 1.1 是執行一個 BTSP 的 2 倍近似演算法在 induced subgraph  $G_i$ ，而執行完步驟 1.2 將會形成一個 path 連接  $G$  中的所有節點且節點  $v_s$  及  $v_d$  會是樹葉，因此得到的生成樹  $T'$  會 BSIMSTP 的一個合理解，而在步驟 2 我們找出所有  $T'$  中權重最小的一棵生成樹。

在第二章中我們證明 BSIMSTP 是 NP-complete，所以目前無法設計多項式時間的正確演算法找到此問題的最佳解 (optimal solution)，除非  $P=NP$ 。然而我們設計的啟發式演算法對於 BSIMSTP，可以在  $O(|V|^4)$  time 找到一組合理解。雖然找到的解不是最佳解，但實驗結果顯示我們的啟發式演算法的近似率會低於 2。

在我們的演算法中，第 1 步驟，我們最多有  $O(|V \setminus R|^2)$  的節點對。在 1.1 步驟，利用 BTSP 的兩倍近似演算法找出一個 TSP 需要花  $O(|V|^2)$  [21]。步驟 1.2 只需花  $O(|V|)$  time。因此根據步驟 1 我們會有最多  $O(|V|^2)$  個生成樹，每個生成樹需要花  $O(|V|^2)$  time 建置。因此針對 BSIMSTP，我們的演算法將可在  $O(|V|^4)$  time 完成。

## 4 實測驗證

本章中，我們將針對第三章所設計的演算法來進行實際的程式模擬。我們藉由 brute force 的正確演算法(exact algorithm)與我們的演算法進行時間效率的分析，我們也會觀察我們的演算法所產生的解的權重與最佳解的權重之間的倍數 (近似率) 關係。近似率(ratio)是拿我們的演算法所得出的解的權重除以最佳解的權重。本次模擬實驗使用 Eclipse 撰寫 Java 程式，作業系統平臺為 64 位元 Microsoft Windows 7 SP1。處理器使用 Intel Core i5-3450，時脈 3.1GHz，記憶體為 8GB。分別以亂數產生 5 個節點與 10 個節點之完全圖 (權重符合 metric) 並用二維陣列儲存無向完全圖之節點及其權重。第一小節中，我們模擬的情形是透過亂數產生器產生 7 個節點的完全圖。第二小節中，我們模擬的情形是透過亂數產生器產生 8 個節點的完全圖。

### 4.1 實測節點數為 7 時的案例

本小節中我們實測當輸入的圖有 7 個節點時我們的啟發式演算法與正確演算法(exact algorithm)所產生的最佳解之間的關係。正確演算法是利用暴力法窮舉所有的生成樹得到 SIMSTP 的最佳解。模擬的情形再分內部節點設定為 1 到 5 個節點的情形，每個實驗分別用亂數產生 50 個隨機的完全圖並將執行後的結果取平均。

為了說明我們的演算法所產生的解與最佳解之間並不會相距太遠，表 1 列出了我們的啟發式演算法所產生出的解與最佳解之間的近似率(ratio)。實驗結果顯示我們的演算法的近似率十分接近 1。因此我們的啟發式演算法在節點數為 7 時可得到相當好的結果。雖然我們未能以

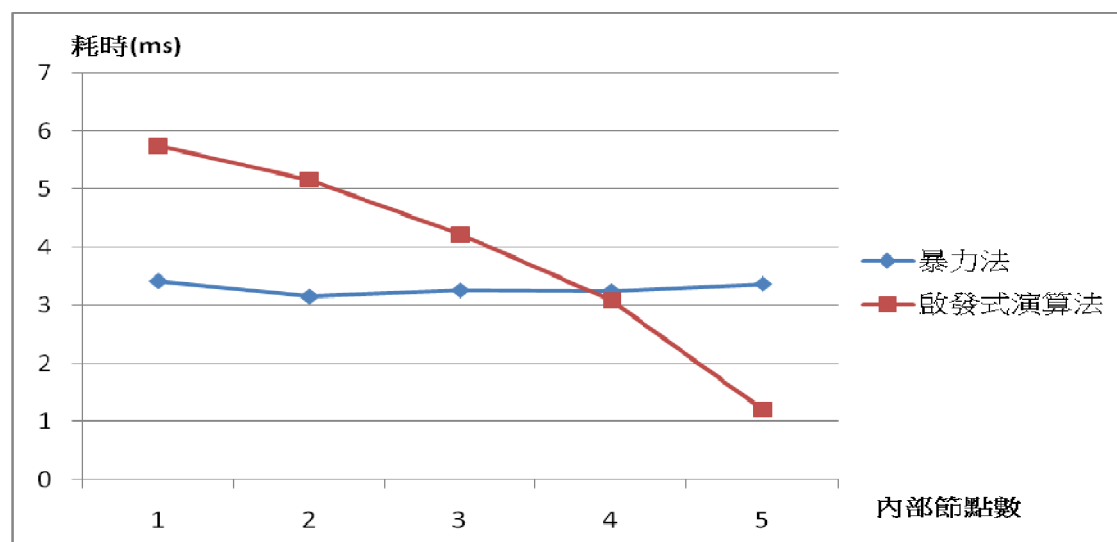


圖 1：在輸入的圖為 7 個節點下，我們的啟發式演算法與暴力法之耗時比較（單位：毫秒）。每個點的值是執行 50 組資料後的平均值。

數學證明我們的解距離最佳解的倍率，然而實驗數據顯示我們的解不失為一個好的結果。

在時間分析上，我們同樣用分析近似率的圖來觀察我們的演算法與正確演算法在執行時的效率。圖 1.列出了我們的演算法與正確演算法在效率上的比較結果。在 7 個節點時兩種演算法執行時間平均差異不大，橫座標是內部節點的個數，縱座標是執行的時間(毫秒)。在內部節點為 1 時，我們的演算法在執行上平均需花 5.752 毫秒，而正確演算法平均需花 3.419 毫秒，在內部節點為 5 時，我們的演算法平均需花 1.205 毫秒，而正確演算法平均需花 3.373 毫秒。實驗結果顯示，我們的演算法會隨內部節點多寡影響執行速度。

| 內部節點數       | 1     | 2     | 3     | 4     | 5     |
|-------------|-------|-------|-------|-------|-------|
| 近似率 (ratio) | 1.397 | 1.328 | 1.340 | 1.221 | 1.150 |
| 標準差         | 0.154 | 0.146 | 0.144 | 0.127 | 0.113 |

表 1. BSIMSTP 的近似率模擬結果(找出最佳解與我們的解的差異)，針對輸入為 7 個節點時情形。ratio 為我們的解的權重除以最佳解的權重。每個數據的值是執行 50 組資料後的平均值。

## 4.2 實測節點數為 8 時的案例

本小節中我們實測當輸入的圖有 8 個節點時我們的啟發式演算法與正確演算法所產生的最佳解之間的關係。內部節點分別設定為 1 到 6 個節點，每個實驗亦是分別用亂數產生 50 個隨機的完全圖並將執行後的結果取平均。

表 2.列出了我們的啟發式演算法所產生出的解與最佳解之間的近似率(ratio)。實驗結果顯示我們的演算法當內部節點數較少時，近似率較大；隨著內部節點的增加，近似率會逐漸趨近 1。當內部節點數少時，近似率平均約 1.4 倍；但是增加到 3 或 6 個內部節點時，就很少出現那麼高的近似率。因此，我們觀察的結果發現，此種啟發式演算法較適合用在內部節點多的情況下。

在時間分析上，我們同樣用分析近似率的圖來觀察我們的演算法與正確演算法在執行時的效率。圖 2. 列出了我們的演算法與正確演算法在效率上的比較結果。在 8 個節點時我們的演算法時間明顯低於正確演算法，這應該是很直覺的，因為正確演算法的時間複雜度成指數成長。橫座標是內部節點的個數，縱座標是執行的時間(毫秒)。在內部節點為 1 時，我們的演算法平均需花 5.518 毫秒，而正確演算法平均需花 48.304 毫秒，在內部節點為 3 時，我們的演算法平均需花 4.491 毫秒，而正確演算法平均需花 37.047 毫秒。在內部節點為 5 時，我們的演算法平均需花 2.828 毫秒，而正確演算法平均需花 34.922 毫秒。實驗結果顯示，我們的演算法受內部節點多寡影響。

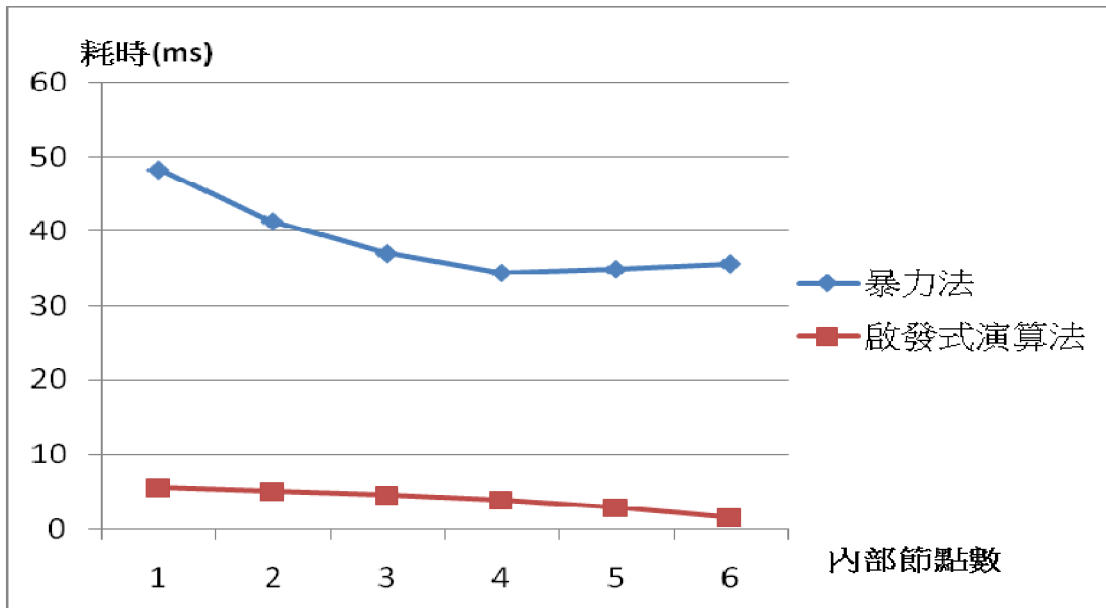


圖 2：在輸入的圖為 8 個節點下，我們的啟發式演算法與暴力法之耗時比較（單位：毫秒）。每個點的值是執行 50 組資料後的平均值。

| 內部<br>節點<br>數      | 1     | 2     | 3     | 4     | 5     | 6     |
|--------------------|-------|-------|-------|-------|-------|-------|
| 近似<br>率<br>(ratio) | 1.429 | 1.395 | 1.328 | 1.319 | 1.301 | 1.174 |
| 標準<br>差            | 0.129 | 0.101 | 0.132 | 0.141 | 0.125 | 0.107 |

表 2. BSIMSTP 的近似率模擬結果(找出最佳解與我們的解的差異)，針對輸入為 8 個節點時情形。ratio 為我們的解的權重除以最佳解的權重。每個數據的值是執行 50 組資料後的平均值。

## 5 未來研究方向

本研究探討瓶頸選擇內部節點最小生成樹問題 (BSIMSTP)，我們證明此問題為 NP-Complete 並對此問題提出一個啟發式演算法。透過實驗也顯示我們的演算法之執行近似率上大約為 1.3 至 1.4 之間，最高到 1.48 倍，低於 1.5 倍，而在執行的效率上是可接受的。未來我們希望能找出理論證明我們的演算法有一個低於 2 倍的近似率，並且設計一個正確的演算法，其時間可以比暴力演算法來得快。

## Acknowledgements

This work was supported in part by the National Science Council of the Republic of China under Contract NSC 101-2815-C-133-008-E

## Reference

- [1] P. Berman and V. Ramaiyer, Improved approximations for the Steiner tree problem., *Journal of Algorithms*, Vol. 17, pp. 381–408, 1994.
- [2] R. Burkard, M. Dell'Amico and S. Martello, *Assignment Problem*, Society for Industrial and Applied Mathematics, 2009.
- [3] J. Byrka, F. Grandoni, T. Rothvoss and L. Sanita, An improved LP-based approximation for Steiner tree, In: *Proceedings of the 42nd ACM Symposium on Theory of Computing*, pp. 583–592, 2010.
- [4] A. Caldwell, A. Kahng, S. Mantik, I. Markov and A. Zelikovsky, On wirelength estimations for row-based placement, In: *Proceedings of the 1998 International Symposium on Physical Design*, pp. 4–11, 1998.
- [5] M. Charikar, J. Naor and B. Schieber, Resource optimization in QoS multicast

- routing of real-time multimedia, *IEEE/ACM Transactions Networking*, Vol. 12, pp. 340–348, 2004.
- [6] X. Cheng and D.Z. Du, *Steiner Tree in Industry*. Kluwer Academic Publishers, Dordrecht, Netherlands, 2001.
- [7] N. Christofides, Worst-case analysis of a new heuristic for the travelling salesman problem, *Report 388*, Graduate School of Industrial Administration, Carnegie Mellon University, 1976.
- [8] T.H. Cormen, C.E. Leiserson, R.L. Rivest and C. Stein, *Introduction to Algorithm*, 2nd edition, MIT Press, Cambridge, 2001.
- [9] D.Z. Du, J.M. Smith and J.H. Rubinstein, *Advances in Steiner tree*, Kluwer Academic Publishers, Dordrecht, Netherlands, 2000.
- [10] D.Z. Du and X. Hu, *Steiner Tree Problems in Computer Communication Networks*, World Scientific Publishing Company, 2008.
- [11] M.R. Garey, R.L. Graham, D.S. Johnson, The complexity of computing Steiner minimal trees, *SIAM Journal of Applied Mathematics*, Vol. 32, pp. 835–859, 1977.
- [12] S.L. Hakimi, Steiner’s problem in graphs and its implications, *Networks*, Vol. 1, pp. 113–133, 1971.
- [13] S. Hougardy and H.J. Prommel, A 1.598 approximation algorithm for the Steiner problem in graphs, In: *Proceedings of the 10th Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 448–453, 1999.
- [14] S.Y. Hsieh and S.C. Yang, Approximating the selected-internal Steiner tree, *Theoretical Computer Science*, Vol. 381, pp. 288–291, 2007.
- [15] F.K. Hwang, D.S. Richards and P. Winter, *The Steiner Tree Problem*, *Annals of Discrete Mathematics*, Vol. 53, Elsevier Science Publishers, Amsterdam, 1992.
- [16] A.B. Kahng and G. Robins, *On Optimal Interconnections for VLSI*, Kluwer Publishers, 1995.
- [17] H.W. Kuhn, The Hungarian method for the assignment problem, *Naval Research Logistics*, Vol. 52, pp. 7–21, 2006.
- [18] A.Y. Levin, Algorithm for shortest connection of a group of graph vertices, *Soviet Mathematics Dokl*, Vol. 12, pp. 1477–1481, 1971.
- [19] X. Li, F. Zou, Y. Huang, D. Kim and W. Wu, A better constant-factor approximation for selected-internal Steiner minimum tree, *Algorithmica*, Vol. 56, pp. 333–341, 2010.
- [20] C.-S. Liu, Y.T. Kuo and C.W. Ma, A heuristic algorithm for the selected-internal minimum spanning tree problem. The 29th Workshop on Combinatorial Mathematics and Computational Theory, pp. 267–271, 2012.
- [21] C.L. Lu, C.Y. Tang and R.C.T. Lee, The full Steiner tree problem, *Theoretical Computer Science*, Vol. 306, pp. 55–67, 2003.
- [22] R.G. Parker and R.L. Rardin, Guaranteed performance heuristics for the bottleneck traveling salesman problem. *Operations Research Letters* Vol. 2, pp. 269–272, 1984.
- [23] H.J. Prommel and A. Steger, A new approximation algorithm for the Steiner tree problem with performance ratio 5/3, *Journal of Algorithms*, Vol. 36, pp. 89–101, 2000.
- [24] G. Robins and A. Zelikovsky, Tighter bounds for graph Steiner tree approximation, *SIAM Journal on Discrete Mathematics*, Vol. 19, pp. 122–134, 2005.
- [25] V.V. Vazirani, *Approximation Algorithms*, Springer-Verlag, Berlin, 2001.
- [26] A. Zelikovsky, An 11/6-approximation algorithm for the network Steiner problem, *Algorithmica*, Vol. 9, pp. 463–470, 1993.
- [27] A. Zelikovsky, A faster approximation algorithm for the Steiner tree problem in graphs, *Information Processing Letters*, Vol. 46, pp. 79–83, 1993.
- [28] A. Zelikovsky, Better approximation bounds for the network and Euclidean Steiner tree problems, *Technical report CS-96-06*, University of Virginia, 1996.